

Università degli Studi di Milano
Dipartimento di Informatica
Mathematical Logic – 2025/2026
Oldani Mattia [53690A]

Binary Decision Diagram



Indice

1. Introduzione	3
2. Motivazioni	3
3. Binary Decision Diagram	3
4. Operazioni sui BDD	4
4.1. Reduce e Reduced BDD	4
4.2. Apply	4
4.3. Restrict	5
4.4. Quantificatori esistenziali e universali	5
5. Implementazione	6
5.1. DummyBDD	6
5.2. SteroidBDD	6
5.2.1. Antlr4	7
5.2.2. Parsed	7
5.2.3. Interactive	8

1. Introduzione

Il problema di decidere la **soddisfacibilità** di una formula della **logica proposizionale** è centrale in numerose applicazioni dell'informatica, insieme ai problemi di **validità** ed **equivalenza logica**.

I **Binary Decision Diagrams (BDD)** costituiscono una delle **tecniche** più diffuse per il calcolo simbolico su formule proposizionali. Un **BDD** è una **struttura dati** che rappresenta la semantica di una formula mediante un grafo diretto aciclico. La formula non viene trattata come una semplice espressione sintattica, ma come una funzione booleana, rappresentata attraverso una struttura grafica che può essere progressivamente ridotta.

Attraverso un **algoritmo di riduzione** il grafo viene trasformato in una forma compatta con una proprietà fondamentale: formule logicamente equivalenti producono lo **stesso BDD ridotto**. Questa caratteristica fornisce una procedura decisionale per l'equivalenza logica: date due formule A_1 e A_2 è sufficiente costruire i rispettivi BDD e verificarne l'identità strutturale.

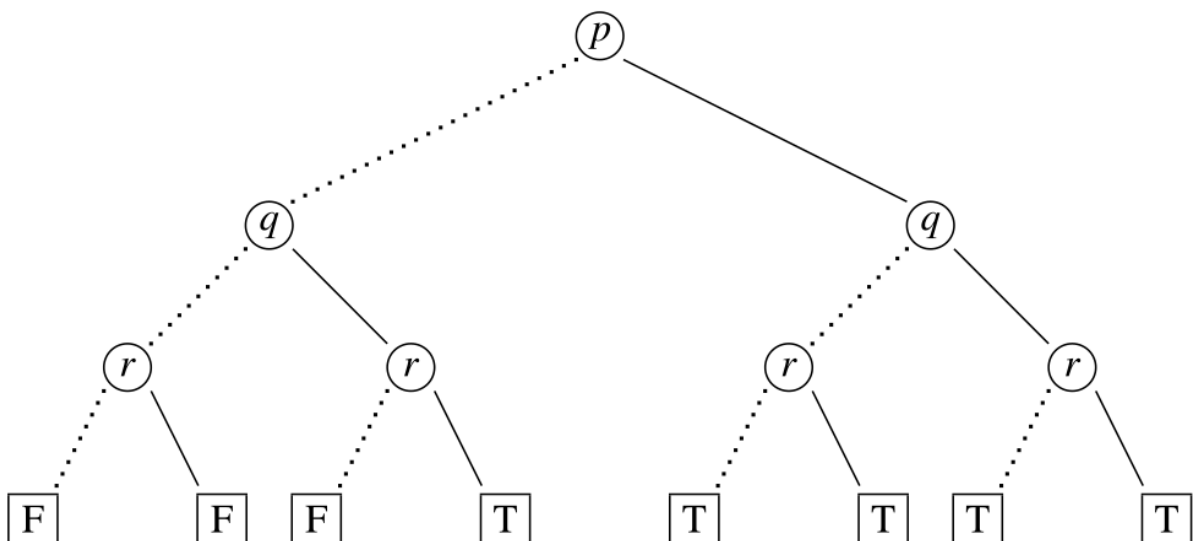
2. Motivazioni

Supponiamo di voler decidere se due formule A_1 e A_2 della logica proposizionale siano **logicamente equivalenti**. Un approccio naturale consiste nel costruire e confrontare le **tabelle di verità** delle due formule. Chiaramente, questo approccio è molto **inefficiente**, perché per ogni formula con n variabili sono necessarie 2^n righe.

Qui entrano in gioco i **BDD**. Il loro uso permette di **ridurre** le tabelle di verità, rimuovendo la **ridondanza** e fornendo una procedura di decisione efficiente per l'equivalenza logica e la soddisfacibilità delle formule.

3. Binary Decision Diagram

Un **Binary Decision Diagram (BDD)**, come una tabella di verità, rappresenta i valori di una formula proposizionale sotto tutte le possibili interpretazioni. Possiamo rappresentare un BDD con un **grafo**, in cui ogni **nodo** è etichettato con una **atomica** e gli **archi** rappresentano l'assegnazione di valori veri o falsi a quella atomica: ad esempio, un arco solido per il valore vero e un arco tratteggiato per il valore falso.



Ogni cammino dall'origine a una foglia corrisponde a un'**interpretazione** completa della formula, e la foglia stessa è etichettata con il valore della formula sotto quell'interpretazione. Notiamo inoltre che nessun atomo compare più di una volta lungo un ramo dall'origine alla foglia, così che venga fatto un solo assegnamento.

A ciascun ramo b è associata un'**interpretazione** I_b . Se il ramo segue l'arco vero di un nodo etichettato p , allora $I_b(p) = T$; se invece prende l'arco falso, allora $I_b(p) = F$. La foglia finale indica il valore $v_{I_b}(A)$ della formula A sotto quell'interpretazione.

La definizione di BDD però non impone un ordine degli atomi lungo i rami. Tuttavia, gli algoritmi sui BDD richiedono che gli ordini lungo i diversi rami non siano in conflitto.

Un **Ordered Binary Decision Diagram (OBDD)** è un BDD in cui l'insieme degli ordini di apparizione degli atomi lungo i rami è compatibile.

Questa piccola estensione ci permetterà dopo di confrontare BDD per diverse formule.

4. Operazioni sui BDD

Possiamo modificare la struttura di un albero tramite una serie di **riduzioni**, che trasformano l'albero fino alla sua versione **ridotta**.

4.1. Reduce e Reduced BDD

L'operazione di riduzione prende in input un **BDD** e crea appunto un **BDD ridotto**.

Vengono effettuate due operazioni:

1. **rimozione delle foglie duplicate**: il BDD ha 2^n foglie, etichettate con **T** e **F**; se il BDD ha più di due foglie distinte, allora rimuoviamo le 2^n foglie aggiungendo invece 2 sole foglie, etichettate con **T** ed **F**, e colleghiamo i nodi che hanno perso le foglie a queste due facendo l'ultimo assegnamento della variabile del nodo;
2. **riduzione ricorsiva**: visitiamo in profondità l'albero partendo dalla radice, e ogni volta che non siamo in nodo n che non è una foglia (caso base della ricorsione) effettuiamo la riduzione sui due sotto-alberi del nodo, che indichiamo con i due nodi radice r_1 ed r_2 . Otteniamo poi due casi:
 - se $r_1 = r_2$ allora ritorniamo r_1 perché l'assegnamento di n dà lo stesso valore r_1 ;
 - se dopo la generazione della tupla $T = (\text{var}(n), r_1, r_2)$ ci accorgiamo che la stessa tupla già stata osservata in una esecuzione precedente (tramite **memoizzazione**), ritorniamo il nodo che l'aveva generata la prima volta.

Il secondo punto della riduzione ricorsiva «rompe» la struttura ad albero, permettendo ad alcuni nodi di avere più padri, ma questo ci permette di non tenere **sotto-alberi simili** e di potare tutti i rami che non servono.

Il BDD che otteniamo dalla **riduzione** è **logicamente equivalente** al BDD di partenza.

Per effettuare poi il confronto tra BDD ridotti ci serve un **ordinamento**: con questa proprietà noi siamo in grado di confrontare tranquillamente due BDD perché li rendiamo **canonici** secondo l'ordinamento dato.

4.2. Apply

Creare un BDD partendo da un **albero completo** è poco conveniente, perché la dimensione iniziale sarebbe simile a quella della tabella di verità – anzi, è anche di più in termini di numero di nodi. La forza dei BDD risiede nella possibilità di **effettuare operazioni** direttamente su **BDD**

ridotti. Infatti, possiamo ottenere un BDD ridotto per l'espressione $A_1 \text{ op } A_2$ conoscendo i BDD ridotti per A_1 e A_2 .

Sfruttiamo ancora un'operazione ricorsiva:

1. se i due alberi sono entrambi foglie etichettate con a_1 e a_2 allora ritorniamo il nodo etichettato da $a_1 \text{ op } a_2$;
2. se i due alberi sono etichettati dalla stessa variabile p ritorniamo un nuovo albero con:
 - p come radice;
 - come sotto-albero SX questa operazione ricorsivamente applicata ai due sotto-alberi SX;
 - idem per il sotto-albero DX;
3. se i due alberi sono etichettati da due variabili p_1 e p_2 tali che $p_1 < p_2$ nell'ordinamento definito, allora ritorniamo un albero con:
 - p_1 come radice;
 - come sotto-albero SX questa operazione ricorsivamente applicata al sotto-albero SX di A_1 e sull'intero albero di A_2 ;
 - idem per il sotto-albero DX;
4. se invece $p_2 < p_1$ stessa cosa ma con A_1 e A_2 invertite.

Questa tecnica ci permette di costruire il **BDD ridotto** senza passare dal **BDD completo**.

4.3. Restrict

L'operazione di **restrizione** prende una formula A , una atomica p e un valore di verità $w \in \{T, F\}$ e restituisce la formula ottenuta sostituendo p con w , effettuando quindi una **valutazione parziale** di A . Come notazione usiamo $A|_{p=w}$.

L'algoritmo di **Restrict** è ancora **ricorsivo** su un BDD dato in input assieme all'atomica p e al valore di verità w :

1. se l'albero è una foglia T oppure F essa viene ritornata;
2. se la radice è p ritornare il BDD che ha come radice il nodo che si raggiunge da p sull'arco definito da w ;
3. se la radice non è p ritornare il nodo etichettato da p e che ha come nodi figli i nodi ottenuti dalla Restrict ricorsiva sui due figli.

Il BDD risultante da Restrict potrebbe non essere ridotto, quindi normalmente si applica subito una operazione di **Reduce**.

4.4. Quantificatori esistenziali e universali

Grazie alla **Restrict** ora possiamo definire i due **quantificatori** principali:

1. **esistenziale** $\exists p(A)$ che vale **T** se A è vera per qualche assegnamento parziale di p ;
2. **universale** $\forall p(A)$ che vale **T** se A è vera su tutti gli assegnamenti parziali di p .

Con i BDD possiamo eseguire due operazioni di **Restrict** per calcolare questi due quantificatori. In particolare:

1. applicando due Restrict, entrambe su p ma una con **T** e una con **F**, e poi applicando una Apply con **OR** otteniamo l'**esistenziale**;
2. applicando due Restrict, entrambe su p ma una con **T** e una con **F**, e poi applicando una Apply con **AND** otteniamo l'**universale**.

Queste operazioni consentono di manipolare formule complesse in maniera efficiente utilizzando **BDD ridotti**, garantendo al contempo una **canonicità** utile per i confronti.

5. Implementazione

L'implementazione Python dei **BDD** ha visto la creazione di una interfaccia **BDD** e di una classe **Node** per la gestione di tutte le operazioni che sono implementabili.

I **Node** modellano i singoli nodi dei BDD, e contengono informazioni sull'atomica che etichetta il nodo, i due nodi figli, il valore di verità (solo se il nodo è foglia) e alcuni dati che permettono di calcolare gli assegnamenti delle foglie.

L'interfaccia **BDD** definisce una serie di operazioni che ogni BDD deve essere capace di eseguire, come la **Restrict**, **Exists**, **Forall**, e altre operazioni di controllo e stampa.

Non sono state inserite **Reduce** e **Apply** come operazioni perché sono stato implementati due BDD diversi, uno **Dummy** che usa la Reduce e uno **Steroid** che usa invece la Apply.

5.1. DummyBDD

Un **DummyBDD** è un BDD che accetta come input una **espressione** e costruisce il BDD completo. Tramite il metodo **reduce** andiamo a ridurre il BDD creato con l'algoritmo di **Reduce** che abbiamo mostrato prima.

```
from alberelli.rbdd.bdd import DummyBDD

expression = 'p | (q & r)'
bdd = DummyBDD(expression)

bdd.print_to_dot('non_reduced')

bdd.reduce()
bdd.print_to_dot('reduced')
```

Ovviamente, questa soluzione è inefficiente visto che viene creato sempre l'albero completo.

Esponiamo comunque i metodi di **Restrict** (restrict), **Exists** (exists) e di **Forall** (forall), ma Exists e Forall non seguono l'implementazione canonica perché non si ha il metodo Apply.

```
from alberelli.rbdd.bdd import DummyBDD

expression = 'p | (q & r)'
bdd = DummyBDD(expression)

bdd.reduce()
bdd.restrict('p', False)

bdd.print_to_dot('restrict')

print(bdd.exists('q'))
print(bdd.forall('q'))
```

5.2. SteroidBDD

Uno **SteroidBDD** è un BDD che rinuncia all'operazione di **Reduce** in favore della **Apply** per costruire un BDD senza passare dall'albero completo. Vedremo **due** diverse **implementazioni** di **SteroidBDD**, che però si basano sulla stessa **grammatica**.

5.2.1. Antlr4

L'idea che hanno gli **SteroidBDD** è quella di costruire il BDD di un'espressione trasformandola in un **Abstract Syntax Tree** (AST), per poi costruire i singoli BDD e infine unirli grazie alla **Apply**. È stata definita una grammatica grazie ad **Antlr4** che si presenta nel seguente modo.

```
grammar PL;

start: expr EOF;

expr: orExpr;

orExpr: xorExpr ('|' xorExpr)*;
xorExpr: andExpr ('^' andExpr)*;
andExpr: notExpr ('&' notExpr)*;
notExpr: 'NOT' '(' expr ')' | atom;
atom: ATOM | '(' expr ')';

ATOM: [a-z]+;
WS: [ \t\r\n]+ -> skip;
```

Con questa grammatica è stata implementata anche la **precedenza**.

5.2.2. Parsed

Una prima versione di SteroidBDD è il **ParsedSteroidBDD**, che accetta come input una **espressione** e costruisce il BDD già ridotto usando la **Apply**. Questo metodo è richiamato da un oggetto **Builder**, generato da **Antlr4**, che visita l'**AST** e mano a mano compone i singoli BDD delle espressioni.

Chiamare più volte il metodo **Apply** su un **ParsedSteroidBDD** non modifica il suo stato.

```
from alberelli.arbdd.bdd import ParsedSteroidBDD

expression = 'p | (q & r)'
bdd = ParsedSteroidBDD(expression)

bdd.print_to_dot('reduced')
```

In questo caso le operazioni di **Exists** e **Forall** sono canoniche perché abbiamo la **Apply**, oltre alla **Restrict**, cruciale per implementare i due quantificatori.

```
from alberelli.arbdd.bdd import ParsedSteroidBDD

expression = 'p | (q & r)'
bdd = ParsedSteroidBDD(expression)

bdd.restrict('p', False)

bdd.print_to_dot('restrict')

print(bdd.exists('q'))
print(bdd.forall('q'))
```

5.2.3. Interactive

Una seconda versione di SteroidBDD è l'**InteractiveSteroidBDD**, che non ha input perché permette di costruire il BDD in maniera **interattiva**, quindi definendo mano a mano le singole variabili e combinandole con le operazioni a nostra disposizione.

Qua, ovviamente, l'AST non va navigato con il **Builder**, non avendo subito una espressione, ma è l'utente che si deve responsabilizzare per effettuare una costruzione corretta della sua espressione.

In questo caso dobbiamo chiamare esplicitamente il metodo **Apply** per costruire il BDD dell'espressione finale.

```
from alberelli.arbdd.bdd import InteractiveSteroidBDD

bdd = InteractiveSteroidBDD()

p = bdd.variable('p')
q = bdd.variable('q')
r = bdd.variable('r')

qANDr = bdd.apply('AND', q, r)
_ = bdd.apply('OR', p, qANDr)

bdd.print_in_dot('reduced')
```

Anche in questo caso le operazioni di **Exists** e **Forall** sono canoniche perché abbiamo la **Apply** e la **Restrict** implementate.

```
from alberelli.arbdd.bdd import InteractiveSteroidBDD

bdd = InteractiveSteroidBDD()

p = bdd.variable('p')
q = bdd.variable('q')
r = bdd.variable('r')

qANDr = bdd.apply('AND', q, r)
_ = bdd.apply('OR', p, qANDr)

bdd.restrict('p', False)

bdd.print_to_dot('restrict')

print(bdd.exists('q'))
print(bdd.forall('q'))
```